Hands-On Novel Workshop

Villanelle O'Reilly

Supervised by:
Marc Hanheide & Georgios Leontidis
Further advice from:
James Brown & Petra Bosilj

April 1st, 2026

Why use HPC?

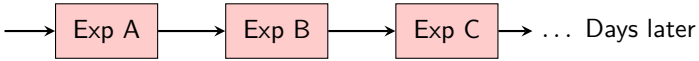
Exploration

- **Problem:** Individual compute forces sequential experimentation (Idea A → Wait → Idea B).
- **Solution:** Simultaneous exploration of the solution space.
- **Impact:** Fail faster to succeed sooner—make good use of the shared resources without submitting ill-considered experiments.

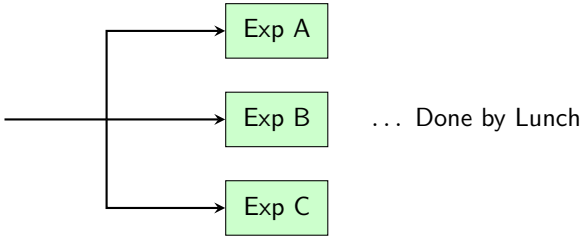
Scale of Experiments

- **Hardware:** Access to High-Mem GPUs and TPUs.
- **Advantage:** Large Batch Sizes.
  - *Speed:* Better parallelisation reduces training time.
  - *Quality:* (typically) Stabilises gradients for better convergence in Deep Learning.
  - *Capability:* Models too large to be reasonably trained on individual compute.

Local Workstation (Serial)



Novel Cluster (Parallel)



Node	GPUs	Total CPUs	Total Memory
GPU Nodes			
hpc-novel-gpu01	8×RTX A6000 (48GiB)	112	950 GiB
hpc-novel-gpu02	4×RTX A6000 (48GiB)	56	480 GiB
hpc-novel-gpu03	4×RTX A6000 (48GiB)	32	240 GiB
hpc-novel-gpu04	8×RTX A6000 (48GiB)	256	950 GiB
hpc-novel-gpu05	4×A100 (80GiB)	128	480 GiB
hpc-novel-gpu06	4×V100 (32GiB)	64	950 GiB
hpc-novel-gpu07	2×RTX Pro 6000 (96GiB)	288	720 GiB
hpc-novel-gpu08	2×RTX Pro 6000 (96GiB)	288	720 GiB
CPU Nodes			
hpc-novel-cpu01	-	72	120 GiB
hpc-novel-cpu02	-	72	120 GiB
hpc-novel-cpu03	-	72	120 GiB
hpc-novel-cpu04	-	128	480 GiB

Tooling 1: Reproducibility with Enroot

- **The Challenge:** Installing and managing bespoke libraries (PyTorch versions, CUDA drivers) on shared infrastructure is difficult.
- **The Solution: Enroot.**
- Run Docker images without root privileges.
- **Benefit:** Ensures reproducible research for you, and for your peers.

Workflow

```
# 1. Import
enroot import docker://pytorch/
     pytorch:latest
# 2. Create
enroot create -n my-env pytorch.
     sqsh
# 3. Run
srun --time=03:00:00 --job-name=
     ttytest --gres=gpu:1 --
     container-image=./pytorch.sqsh
      --container-mount-home --pty
     bash
```

Tooling 2: Managing Chaos with Tracking

- When running parallel jobs, you cannot feasibly rely on terminal output alone.
- **Experiment Tracking (ClearML, W&B, Neptune):**
  - Automatically logs environment, Git commit hash, and uncommitted diffs.
  - Reports scalar variables (Loss, Accuracy) live, with plots comparing to previous experiments updating in realtime.
- *Note for non-CS researchers:* Regardless of the discipline, automated tracking removes tedium and guesswork - it is completely impractical for me to manually go through the logs of the 100+ experiments I ran building up to, and in testing work from my recent submitted paper.

Workshop GitHub Repo

Figure 1: https://github.com/LCAS/novel_hpc_workshop/

A Brief Overview of Slurm

- **Slurm** is the cluster's workload manager and job scheduler.
- It allocates compute resources (CPUs, GPUs, Memory) and ensures fair usage among researchers.
- **Key Commands:**
  - `sbatch`: Submit a non-interactive script to run in the background.
  - `srun`: Run an interactive (blocking) job.
  - `squeue`: View the queue of running and pending jobs.
  - `sinfo`: View the status of cluster nodes (idle, allocated, down).
  - `scancel <jobid>`: Terminate one of your active or pending jobs.

Quality of Life Utilities: Installation

- Native Slurm commands can be incredibly verbose. I've created several wrapper scripts to streamline monitoring and interactive development.
- After SSH-ing into the cluster, run this snippet on the **login node** to install them:

Install QoL Utilities

```
mkdir -p ~/.local/bin
cd ~/.local/bin
for cmd in lsgpu lsjob erun epy epip; do
    curl -sL "https://s.vnet.tel/$cmd" -o "$cmd"
    chmod +x "$cmd"
done
echo 'export␣PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Quality of Life Utilities: What they do

- **Monitoring** (Easier alternatives to `sinfo`/`squeue`):
  - `lsgpu`: Auto-resizing, colourised table showing real-time free/total resources (GPUs, CPUs, RAM) per node.
  - `lsjob`: Formats all running/pending jobs into a clean table. Use `lsjob -w` for a live updating view.
- **Interactive Environments** (Wraps `srun` + Enroot):
  - `erun`: Instantly requests a GPU and drops you into a bash shell inside a PyTorch Enroot container. Auto-activates virtual environments.
  - `epy <args>`: Runs Python non-interactively on a GPU node inside the container environment.
  - `epip <args>`: Safely runs `pip` inside your virtual environment on a GPU compute node.

Access to Novel

- You need access to the university network (e.g. https://remote.lincoln.ac.uk)
- If the hostname `login.novel.hpc` fails, use `10.10.33.11` or `10.10.33.12`.
- It's best to use SSH keys, and required for vscode. Generate strong keys per-device for a minimal attack surface to avoid needing to transfer private keys

Access

```
# 0. You should be on the university network
ping -c5 login.novel.hpc

# 1. SSH in
ssh <first_initial><lastname>@login.novel.hpc
# 2. Check slurm
sinfo
# 3. Run a test
srun --time=00:00:05 --job-name=testjobg --
    partition=gpu --gres=gpu:1 nvidia-smi
# 3.5. or
srun --time=00:00:05 --job-name=testjobc --
    partition=cpu lscpu
```

SSH Keys I

Generate Keys & Config On **YOUR Device**

```
# 0. CD to .ssh
voreilly@homedesk:~$ cd ~/.ssh/
# 1. Make a key
voreilly@homedesk:~/.ssh$ ssh-keygen -t rsa -b 4096 -f "id_novelfrom${HOSTNAME}_rsa"
# 2. Check permissions in ~/.ssh/
voreilly@homedesk:~/.ssh$ ls -lah
drwx------   2 voreilly voreilly 4.0K Nov 11 15:28 .
drwx------ 113 voreilly voreilly  12K Mar 31 17:02 ..
-rw-------   1 voreilly voreilly 1.5K Oct 22  2024 authorized_keys
-rw-------   1 voreilly voreilly 3.4K Jul 12  2025 id_novelfromhomedesk_rsa
-rw-r--r--   1 voreilly voreilly  745 Jul 12  2025 id_novelfromhomedesk_rsa.pub
```

SSH Keys II

Generate Keys & Config On **YOUR Device**

```
# 3. Copy your key to Novel
ssh-copy-id -i ~/.ssh/id_novelfrom${HOSTNAME}_rsa.pub <
    username>@login.novel.hpc
# 4. Make or append to your .ssh config
cat <<EOF >> ~/.ssh/config && chmod 600 ~/.ssh/config
Host novel
    HostName login.novel.hpc
    User <username>
    IdentitiesOnly yes
    IdentityFile ~/.ssh/id_novelfrom${HOSTNAME}_rsa
EOF
```

Test SSH Keys

```
# 5. Give it a whirl!
voreilly@homedesk:~/.ssh$ ssh
    novel
[voreilly@hpc-novel-login01 ~]$
```

OCR Demo

- To demonstrate how to use the cluster, we'll train a single-character OCR model.
- The training script spits out an `.onnx` file. Download it via SFTP/VSCode to use with the local web app (`ocr_demo_app.html`) and qualitatively evaluate your model!
- You'll have to use git to clone the repo, and customise `train.sh` to request a GPU for training.

Figure 2: https://ocrdemo.vnet.tel/

OCR Demo: Setup Environment

- First, we need to grab the code and set up a Python virtual environment.
- We use our epy tool to create the environment on a compute node, then activate it on the login node.
- Lastly, we use epip to safely install our packages on the compute node.

Clone & Setup

```
# 1. Clone the repository
git clone https://github.com/LCAS/novel_hpc_workshop.git
cd novel_hpc_workshop
# 2. Create the virtual environment via a compute node
epy -m venv --copies --system-site-packages .venv
# 3. Activate the environment on the login node
source .venv/bin/activate
# 4. Install requirements via a compute node
epip install -r requirements.txt
```

OCR Demo: Submitting & Monitoring

- Once your environment is set up and your `train.sh` script is ready, submit the job.
- Because it runs in the background, output won't print to your terminal directly. We must tail the log files.

Submit & Watch Logs

```
# 1. Submit the script
sbatch train.sh

# 2. Check its status in the queue
lsjob -w

# 3. Watch the live output logs (Ctrl+C to exit)
tail -f logs/*
```

OCR Demo: Resource Utilisation

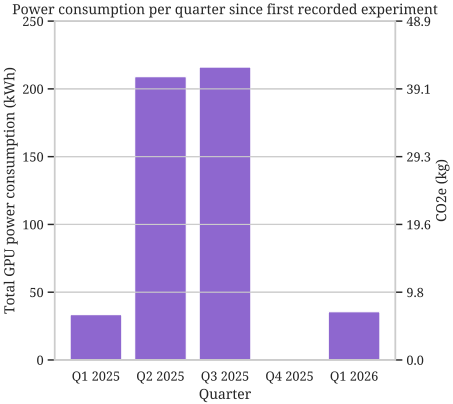
- It's critical to ensure you aren't requesting far too much (or too little) memory, CPU, or GPU resources.
- Use the `sperf` utility followed by your Job ID to check your efficiency.

Check Efficiency with sperf

```
$ sperf 9840
RESOURCE UTILISATION REPORT
Job ID : 9840
Allocated memory : 64.000 GB
Used memory : 16.365 GB
Memory utilisation : 25.57%
CPU utilisation (normalised) : 5.10%
GPU utilisation (normalised) : 32.64%
```

Environmental Impact

- 494.099 kWh recorded GPU energy usage ≈ 96.611 kg of CO2e emissions from generation and transmission. Conversion factors from UK Gov't Department for Energy Security and Net Zero, 2025
- Every experiment since 20th Jan 2025 tracks cumulative GPU energy usage



Conclusion

- Shared compute resources is better for everyone! Especially when the hardware we use has an expiry date.
- Enabled **Breadth**: Parallel experiments for faster validation.
- Enabled **Depth**: Large scale models/batches on TPUs and powerful GPUs.
- **Recommendation:** Containerise early (Enroot) and track everything (W&B).

References I

UK Gov't Department for Energy Security and Net Zero. (2025). Greenhouse gas reporting: conversion factors 2025 [[Online; accessed 07-September-2025]]. https://www.gov.uk/government/publications/greenhouse-gas-reporting-conversion-factors-2025